TP5 – CI5 : Image Captioning avec RNN et Attention sur ResNet

CSC 8607 – Introduction au deep learning

Maroua Zoubir
Télécom SudParis

8 novembre 2026

1 Objectif et architecture

Le but est de générer une légende textuelle à partir d'une image (dataset **Flickr30k**, environ 30 000 images avec 5 légendes chacune). L'architecture combine :

1. un **ResNet50 pré-entraîné** sur ImageNet, gelé, qui fournit une carte de caractéristiques spatiale de forme $(2048, 7, 7)$;
2. des **embeddings Word2Vec** (GoogleNews, dimension 300), gelés ;
3. un **module d'attention spatiale** qui, à chaque pas de temps, pondère les 49 régions de l'image selon l'état caché courant ;
4. une **cellule LSTM écrite à la main**, recevant à chaque pas la concaténation (contexte, mot, h_{t-1}) ;
5. une couche linéaire de projection vers le vocabulaire.

Les équations de l'attention sont :

$$\begin{aligned} \text{scores}_{att} &= W_{att} \cdot \text{concat}(\text{features}, \text{hidden_state}) \\ \text{poids}_{att} &= \text{softmax}(\text{scores}_{att}) \\ \text{context} &= \sum \text{poids}_{att} \times \text{features} \end{aligned}$$

2 Exercice 1 – Chargement des données et embeddings

Le `shuffle=True` sur le train décorrèle les batchs d'une époque à l'autre ; `shuffle=False` sur le test garantit des évaluations reproductibles.

2.1 Question : pourquoi redimensionner toutes les images en $(224, 224)$?

— **Contrainte de batching** : un batch est un tenseur $(B, 3, H, W)$; toutes les images doivent avoir la même taille pour être empilées, alors que Flickr30k contient des résolutions et ratios variés.
— **Compatibilité avec le réseau pré-entraîné** : le ResNet50 a été entraîné sur des images 224×224 avec la normalisation ImageNet. Cette taille donne une carte de sortie de taille fixe 7×7 (224/32), indispensable pour que l'attention manipule toujours 49 régions et que l'état caché soit étalé sur une grille connue.
— **Coût mémoire et calcul** : le coût des convolutions croît avec le nombre de pixels ; une taille fixe et modérée rend le temps et la mémoire par batch prévisibles.

2.2 Embeddings Word2Vec (`text_utils.py`)

Le script construit le vocabulaire des légendes, parcourt le fichier binaire Word2Vec et ne conserve que les vecteurs des mots présents dans le vocabulaire pour initialiser une couche `nn.Embedding`. `tokenize` convertit une phrase en indices de longueur fixe (`max_length=20`, complétée par des 0) et `untokenize` effectue l'opération inverse.

3 Exercice 2 – Extraction de caractéristiques et transfer learning

Le slice `[:-2]` retire `AdaptiveAvgPool2d` et `Linear` : on conserve la carte spatiale plutôt qu'un vecteur global.

3.1 Question : pourquoi geler les poids du ResNet ?

— **Raison matérielle** : ResNet50 compte environ 23,5 M de paramètres (hors couche finale). Les entraîner impliquerait de stocker leurs gradients et les états d'Adam (deux moments par paramètre) et de rétropropager à travers 50 couches pour chaque batch 224×224 : mémoire GPU et temps de calcul bien plus élevés. Geler le backbone évite de stocker le graphe de calcul associé.

— **Raison algorithmique** : le ResNet est déjà un excellent extracteur de features visuelles. Le réentraîner avec les gradients bruités d'un décodeur LSTM initialisé aléatoirement risque de dégrader ces représentations (*catastrophic forgetting*) et de provoquer du surapprentissage, Flickr30k étant bien plus petit qu'ImageNet. On n'apprend ainsi que l'attention, le LSTM et la projection de sortie.

3.2 Question : forme du tenseur de sortie

Pour une entrée $(32, 3, 224, 224)$, ResNet50 réduit la résolution d'un facteur 32 ($224/32 = 7$) et le dernier bloc produit 2048 canaux. La sortie est donc :

$$\boxed{(32,\ 2048,\ 7,\ 7)}$$

4 Exercice 3 – Module d'attention et LSTM personnalisé

— `torch.cat(..., dim=1)` : concaténation sur la dimension des **canaux** (2048+hidden_dim).

— `F.softmax(..., dim=1)` : appliqué sur les 49 régions aplaties ; les poids sont positifs et somment à 1 pour chaque image.

— Somme pondérée sur `dim=3` puis `dim=2` (W puis H) : on obtient un vecteur de forme $(\text{batch}, 2048)$.

4.1 Question : que représente `context_vector` ?

Le `context_vector` est un **résumé dynamique de l'image, centré sur les régions les plus pertinentes pour prédire le mot courant** (moyenne des 49 features pondérée par des poids d'attention dépendant de l'état caché du LSTM). Il résout le goulot d'étranglement du Seq2Seq de base, où toute l'information de l'image est comprimée en un unique vecteur global et statique, identique à chaque pas de temps : avec l'attention, le décodeur peut regarder une zone différente de l'image selon le mot généré.

Les portes d'entrée, d'oubli et de sortie utilisent une **sigmoïde** (valeurs dans $[0, 1]$, rôle de vanne), tandis que la cellule candidate g_t et la sortie $\tanh(c_t)$ utilisent un **tanh** (valeurs dans $[-1, 1]$).

5 Exercice 4 – Modèle global et entraînement

Hyperparamètres : Adam, lr $= 10^{-3}$, `StepLR(step_size=5, gamma=0.1)`, `CrossEntropyLoss(ignore_index=0)` (le padding est ignoré), `batch_size=32`, 15 époques, séparation 80 %/20 % (seed 42).

Remarque : le LSTM reçoit les mots décalés (`caps[:, :-1]`) et la cible est `caps[:, 1:]` : c'est le *teacher forcing*.

5.1 Lancement de l'entraînement

L'entraînement a été lancé sur Slurm .

```
[compute] [job 54516] mzoubir@starfighter-slurm-node-01-1:~/TP3/TP5 (tp4-clean-publication)
$ python train.py
/mnt/hdd/homes/mzoubir/miniforge3/envs/deeplearning/lib/python3.10/site-packages/torch/utils/data/data
rWarning: This DataLoader will create 4 worker processes in total. Our suggested max number of worker
is 1, which is smaller than what this DataLoader is going to create. Please be aware that excessive wo
t get DataLoader running slow or even freeze, lower the worker number to avoid potential slowness/free
  self.check_worker_number_rationality()
Computing vocabulary...
```

FIGURE 1 – Lancement de l'entraimnement

```
Epoch 1/15
Train Loss: 23.9247
Validation Loss: 4.6765
Prediction : man men are sitting in on front area with people in in man in other man in on be
Target     : four people are scuba diving in swimming pool two girls one bald man the third person you can only

Epoch 2/15
Train Loss: 20.1715
Validation Loss: 4.3387
Prediction : two men are sitting in in the pool with other one in man in other man in in be
Target     : four people are scuba diving in swimming pool two girls one bald man the third person you can only

Epoch 3/15
Train Loss: 19.8217
Validation Loss: 4.2302
Prediction : two men are sitting in in the pool with other are in man in other man is the be
Target     : four people are scuba diving in swimming pool two girls one bald man the third person you can only

Epoch 4/15
Train Loss: 17.7040
Validation Loss: 4.1935
Prediction : two men are standing underwater device the pool with men are in man in other is is look be
Target     : four people are scuba diving in swimming pool two girls one bald man the third person you can only
```

FIGURE 2 – Resultats

5.2 Résultats de l'entraînement (15 époques)

Époque	LR	Loss train	Loss validation
1	10^{-3}	23,92	4,677
2	10^{-3}	20,17	4,339
3	10^{-3}	19,82	4,230
4	10^{-3}	17,70	**4,194** (minimum)
5	10^{-3}	16,17	4,216
6	10^{-4}	13,89	4,205
10	10^{-4}	13,06	4,243
11	10^{-5}	12,55	4,243
15	10^{-5}	12,78	4,251

TABLE 1 – Évolution des losses et du learning rate.

Remarque : la loss d'entraînement affichée n'est pas sur la même échelle que la loss de validation (environ 3 fois plus grande, vraisemblablement à cause de son agrégation dans la boucle). On compare donc les **tendances** de chaque courbe, pas leurs valeurs absolues.

5.3 Évolution des légendes générées (époque 1 vs époque 10)

Cible (identique à chaque époque) : *« four people are scuba diving in swimming pool two girls one bald man the third person you can only »*

Époque	Prédiction
1	`man men are sitting in on front area with people in in man in other man in on be`
10	`two men are standing underwater in the pool with are are in man in other is is are be`

TABLE 2 – Prédictions aux époques 1 et 10.

Observations.

- **Époque 1** : la phrase n'a quasiment aucune structure. Elle enchaîne des mots très fréquents (« in in », « man … man », « in on be ») et seul « sitting » évoque vaguement une scène. Le modèle n'a pour l'instant appris que la distribution marginale des mots du corpus.
- **Époque 10** : le **début de la phrase s'améliore nettement** : « two men are standing underwater in the pool » est grammaticalement correct et **cohérent avec l'image** (plongeurs sous l'eau dans une piscine). Le modèle se trompe encore sur le nombre et le sexe des personnes (« two men » au lieu de « four people ») et surtout la **fin de la phrase reste incohérente** (« with are are in man in other is is are be ») avec des répétitions de mots-outils.
- Cette dégradation en fin de phrase apparaît **même avec le teacher forcing** : à chaque pas le modèle reçoit le vrai mot précédent et prédit pourtant « are are » ou « is is ». Le modèle est donc encore sous-entraîné pour les positions tardives de la séquence, et ce n'est pas un simple effet d'accumulation d'erreurs de génération.
- **À partir de l'époque 8, la prédiction ne change plus** (seul un mot varie entre les époques 8 et 9), ce qui correspond à un modèle qui a cessé d'apprendre (voir ci-dessous).

Le modèle **s'améliore donc**, surtout sur le début de la légende, mais la syntaxe globale reste loin d'être satisfaisante.

5.4 Impact du planificateur de learning rate (StepLR)

Avec `step_size=5` et `gamma=0.1`, le learning rate vaut 10^{-3} (époques 1–5), 10^{-4} (6–10), puis 10^{-5} (11–15). Les courbes montrent trois phases :

- **Époques 1–5 (lr = 10^{-3})** : la validation baisse vite jusqu'à l'époque 4 (4,677 → 4,194) puis **remonte à l'époque 5** (4,216) alors que la loss d'entraînement continue de diminuer : le modèle commence à surapprendre et le pas de 10^{-3} est déjà trop grand pour affiner la solution.
- **Époque 6 (premier decay, lr = 10^{-4})** : la loss d'entraînement chute d'un coup (16,17 → 13,89) et la validation s'améliore légèrement (4,216 → 4,205). Le decay permet de descendre plus finement dans la vallée de la loss.
- **Époques 7–15 (10^{-4} puis 10^{-5})** : la loss d'entraînement se stabilise (autour de 13,0 puis 12,5–13,1, avec de petites fluctuations dues au bruit des mini-batchs), la validation reste quasi constante autour de 4,24–4,25 et les légendes sont figées. Avec 10^{-5}, les pas sont trop petits pour que le modèle évolue encore.

Que se passerait-il avec un lr constant à 10^{-3} ? Les époques 1 à 5 donnent une indication : le modèle progresse vite puis la validation cesse de baisser et oscille autour de son minimum (4,19 → 4,22). En gardant ce pas jusqu'à l'époque 15, les paramètres continueraient de **rebondir autour du minimum** sans converger, la loss d'entraînement resterait plus bruitée, la validation serait instable d'une époque à l'autre et les légendes changeraient à chaque époque au lieu de se stabiliser ; le risque de surapprentissage serait aussi plus élevé. (Il s'agit d'un raisonnement à partir des courbes : nous n'avons pas relancé d'entraînement à lr constant.)

Pourquoi c'est particulièrement pertinent avec Adam ? Adam combine un moment d'ordre 1 (moyenne mobile des gradients, effet momentum) et un moment d'ordre 2 (normalisation par la racine de la moyenne mobile des carrés des gradients, comme RMSProp). La taille effective du pas par paramètre reste de l'ordre de lr quelle que soit la magnitude du gradient : quand les gradients deviennent petits en fin d'entraînement, la normalisation les amplifie et le pas ne diminue pas de lui-même. Le momentum ajoute de l'inertie qui favorise le dépassement du minimum. Réduire explicitement le learning rate est donc nécessaire pour affiner la convergence, comme le montre la chute de loss à l'époque 6.

Limite observée. Le decay est ici assez agressif (÷10 toutes les 5 époques) : le lr est déjà à 10^{-5} dès l'époque 11, ce qui explique le plateau complet des dernières époques. Un `gamma` plus doux (par exemple 0,5) ou un arrêt anticipé autour des époques 4–6 (meilleure validation) aurait été plus efficace.